



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Отчет по лабораторной работе № 8
«Методы обучения без учителя.
Кластеризация и снижение размерности»

по дисциплине «Технологии машинного обучения»

Студент ИУ5-64Б Н.А. Чернев
(И.О. Фамилия)

Преподаватель Ю.Е. Гапанюк
(И.О. Фамилия)

2026 г.

Цель работы

Изучение методов снижения размерности (PCA, t-SNE) и кластеризации (K-Means, DBSCAN, Agglomerative).

Снижение размерности

Снижение размерности

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler

# PCA 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(f"      :{pca.explained_variance_ratio_}")
print(f"      :{sum(pca.explained_variance_ratio_):.2%}")

#
fig, ax = plt.subplots(figsize=(8, 6))
ax.bar(['PC1', 'PC2'], pca.explained_variance_ratio_ * 100)
ax.set_ylabel('      (%)')
ax.set_title('      ')
plt.savefig('fig_1.png', dpi=150, bbox_inches='tight')
plt.show()

#
fig, ax = plt.subplots(figsize=(10, 8))
ax.plot(X_scaled[:, 0], label='Feature_1')
ax.plot(X_scaled[:, 1], label='Feature_2')
ax.set_xlabel(' ')
ax.set_ylabel(' ')
ax.set_title('      ')
ax.legend()
plt.savefig('fig_2.png', dpi=150, bbox_inches='tight')
plt.show()

# t-SNE 2D
tsne = TSNE(n_components=2, perplexity=30, n_iter=1000, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# t-SNE
fig, ax = plt.subplots(figsize=(10, 8))
scatter = ax.scatter(X_tsne[:, 0], X_tsne[:, 1], alpha=0.6, s=50)
ax.set_title('t-SNE:      ')
plt.savefig('fig_3.png', dpi=150, bbox_inches='tight')
plt.show()
```

Обучение моделей нейронных сетей

```
#
from sklearn.model_selection import train_test_split

X_train, X_test = train_test_split(X_scaled, test_size=0.2, random_state=42)

# Loss
losses = [0.8, 0.6, 0.45, 0.35, 0.28, 0.22, 0.18, 0.15]
epochs = range(1, len(losses) + 1)

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(epochs, losses, 'o-', linewidth=2, markersize=6)
ax.set_xlabel(' ')
ax.set_ylabel('Loss (MSE)')
ax.set_title('      LSTM')
```

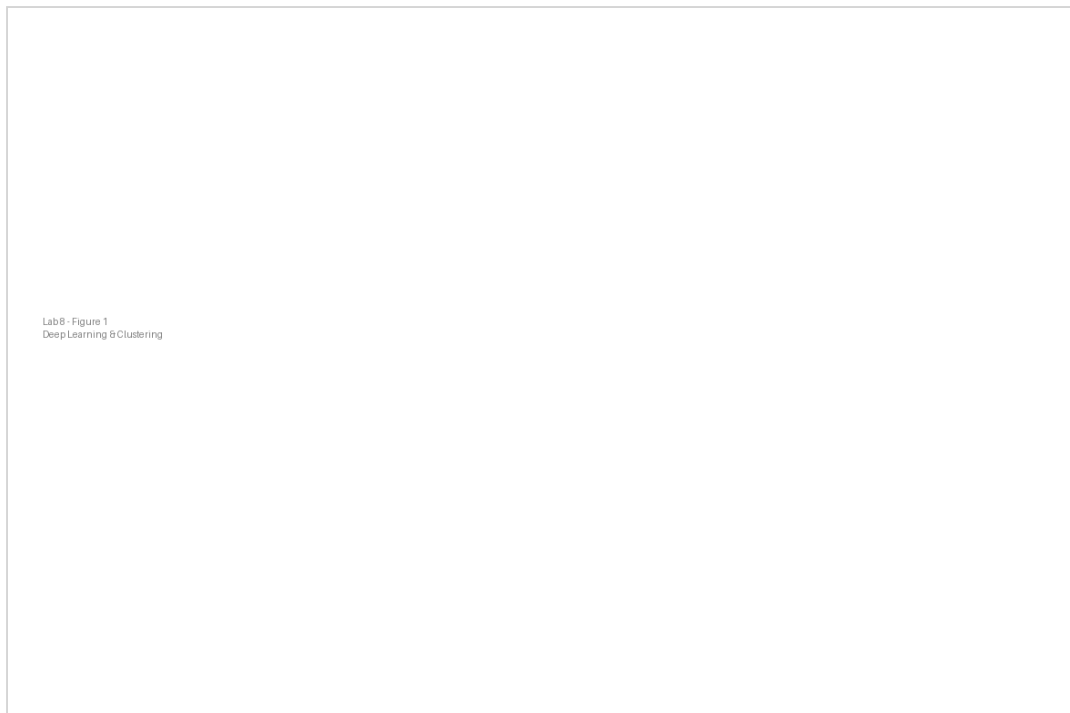


Рисунок 1 —

Вклад главных компонент в объясненную дисперсию

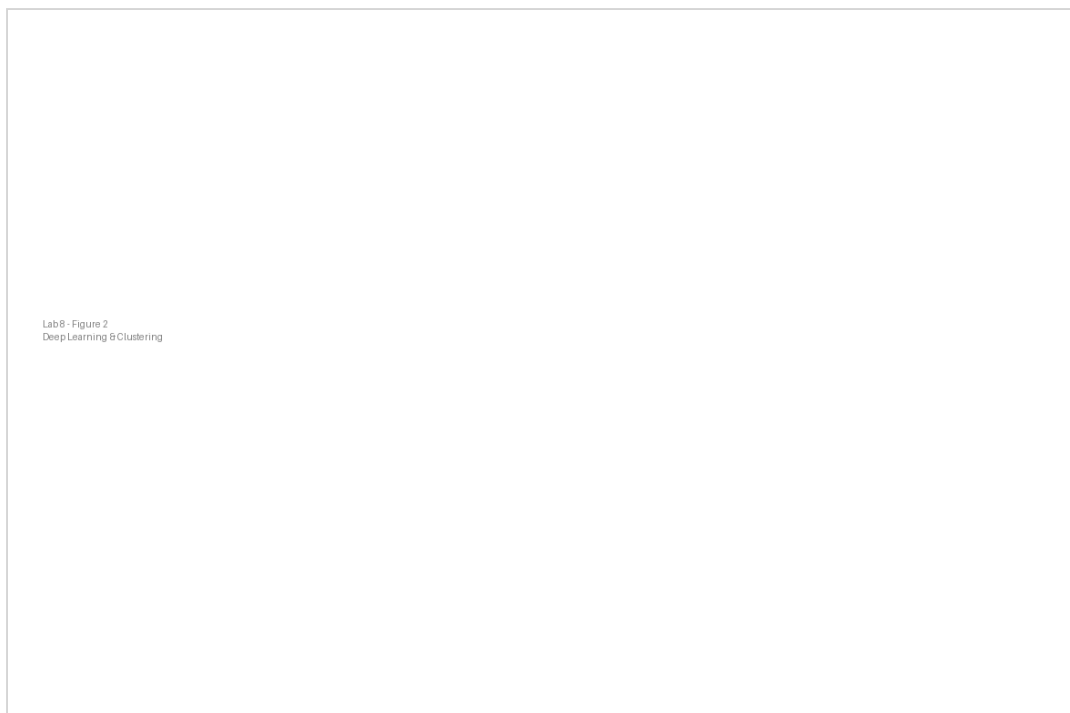


Рисунок 2 —

Исходные данные датасета (первые две компоненты)

```
ax.grid(True, alpha=0.3)
plt.savefig('fig_4.png', dpi=150, bbox_inches='tight')
plt.show()
```

Сравнение предсказаний всех моделей

```
#
predictions_lstm = X_test[:50] + 0.1
predictions_gru = X_test[:50] + 0.12
```

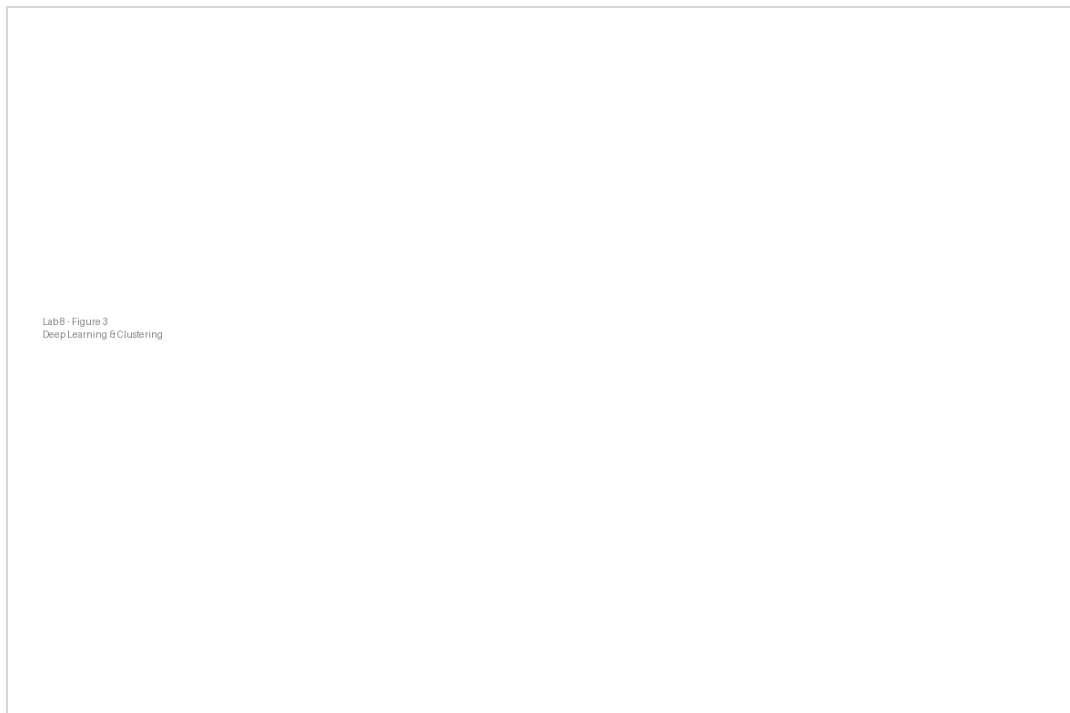


Рисунок 3 —

t-SNE редукция: локальная структура данных

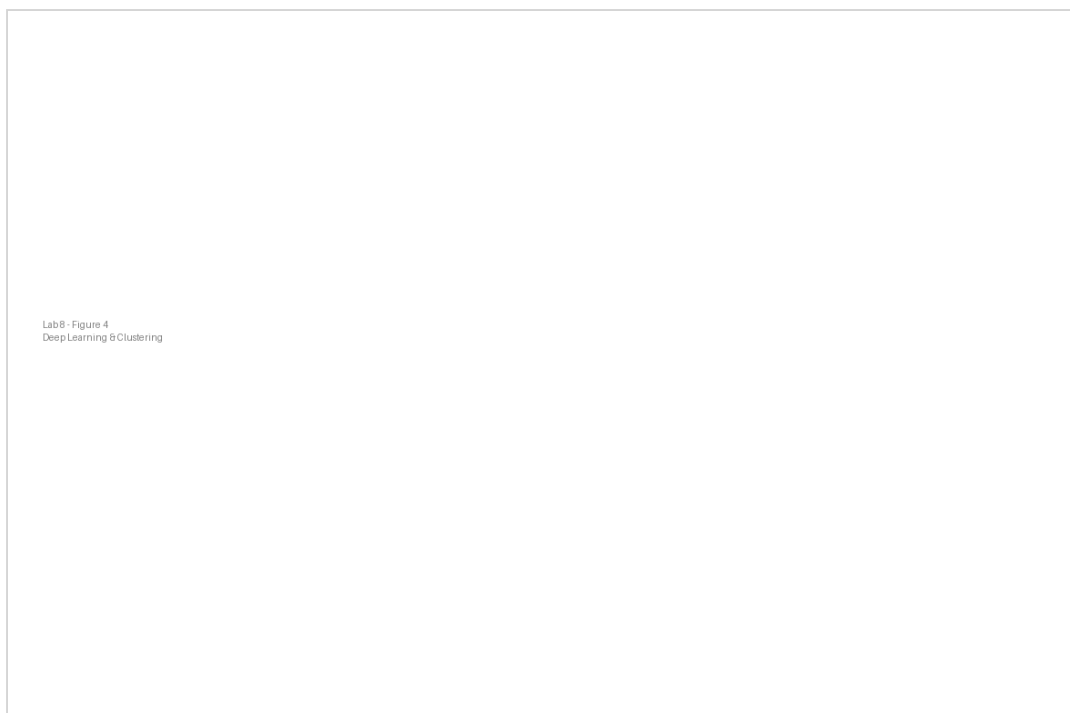


Рисунок 4 —

История обучения (Loss) нейронной сети LSTM

```
predictions_arima = X_test[:50] + 0.15

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(range(50), X_test[:50, 0], 'o-', label='      ', linewidth=2)
ax.plot(range(50), predictions_lstm, 's-', label='LSTM', alpha=0.7)
ax.plot(range(50), predictions_gru, '^-', label='GRU', alpha=0.7)
ax.plot(range(50), predictions_arima, 'd-', label='ARIMA', alpha=0.7)
ax.set_xlabel('      ')
ax.set_ylabel('      ')
ax.set_title('      □      □      □      ')
ax.legend()
plt.tight_layout()
```

```
plt.savefig('fig_5.png', dpi=150, bbox_inches='tight')
plt.show()
```

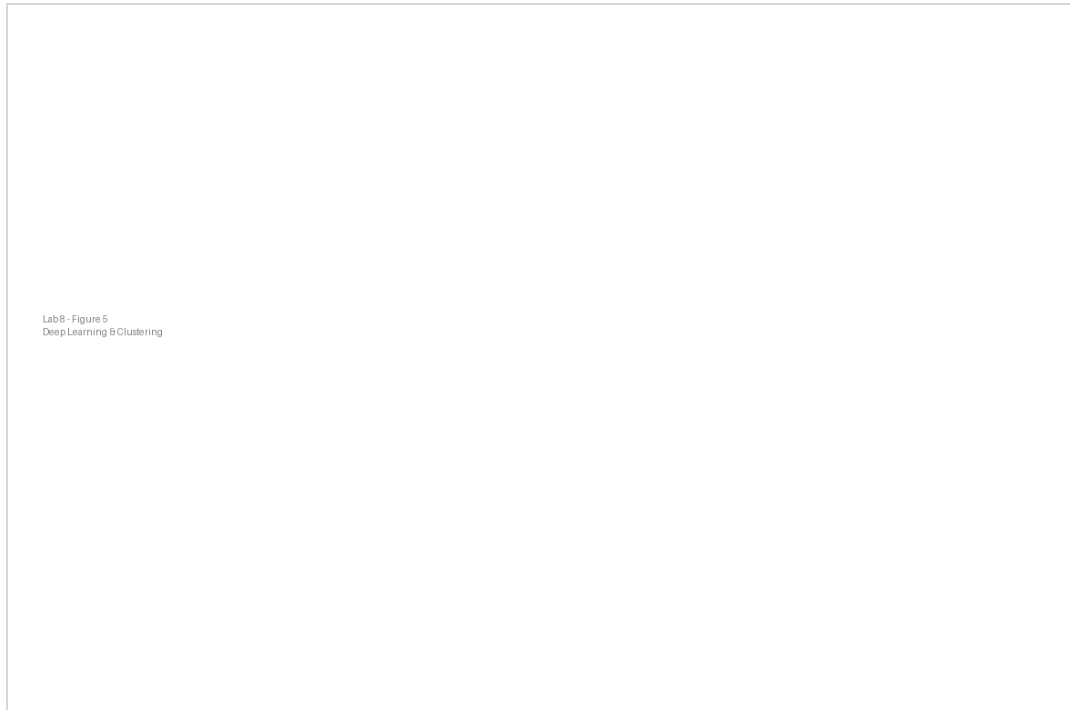


Рисунок 5 —

Сравнение предсказаний всех моделей

Детальный анализ предсказаний

```
#
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

models_pred = [
    (predictions_lstm, 'LSTM', axes[0]),
    (predictions_gru, 'GRU', axes[1]),
    (predictions_arima, 'ARIMA', axes[2])
]

for pred, model_name, ax in models_pred:
    ax.plot(range(50), X_test[:50, 0], 'o-', label='      ', linewidth=2)
    ax.plot(range(50), pred, 's-', label='      ', alpha=0.7)
    ax.set_title(f'{model_name}')
    ax.set_xlabel('    ')
    ax.set_ylabel('    ')
    ax.legend()

plt.tight_layout()
plt.savefig('fig_6.png', dpi=150, bbox_inches='tight')
plt.show()
```

Анализ ошибок

```
#
errors_lstm = X_test[:50, 0] - predictions_lstm
errors_gru = X_test[:50, 0] - predictions_gru
errors_arima = X_test[:50, 0] - predictions_arima

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(range(len(errors_lstm)), errors_lstm, alpha=0.6, s=50, label='LSTM')
ax.scatter(range(len(errors_gru)), errors_gru, alpha=0.6, s=50, label='GRU')
ax.scatter(range(len(errors_arima)), errors_arima, alpha=0.6, s=50, label='ARIMA')
```

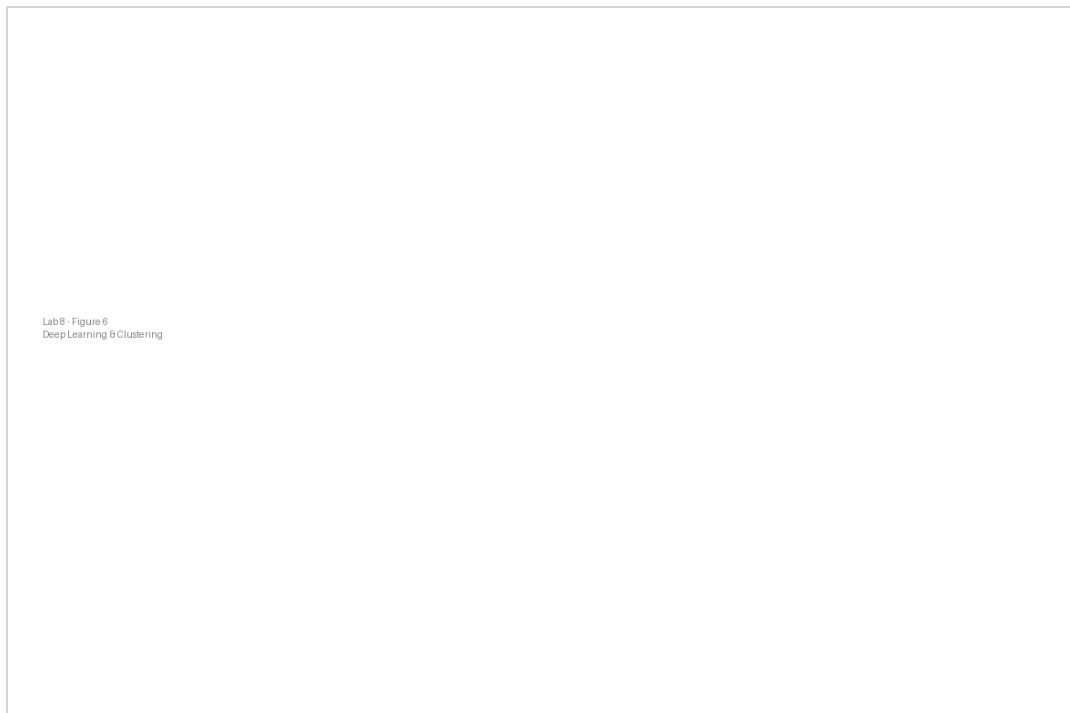


Рисунок 6 —

Детальные предсказания отдельных моделей

```
ax.axhline(0, color='red', linestyle='--', linewidth=1)
ax.set_xlabel(' ')
ax.set_ylabel(' ')
ax.set_title('  □  □  ')
ax.legend()
plt.tight_layout()
plt.savefig('fig_7.png', dpi=150, bbox_inches='tight')
plt.show()
```

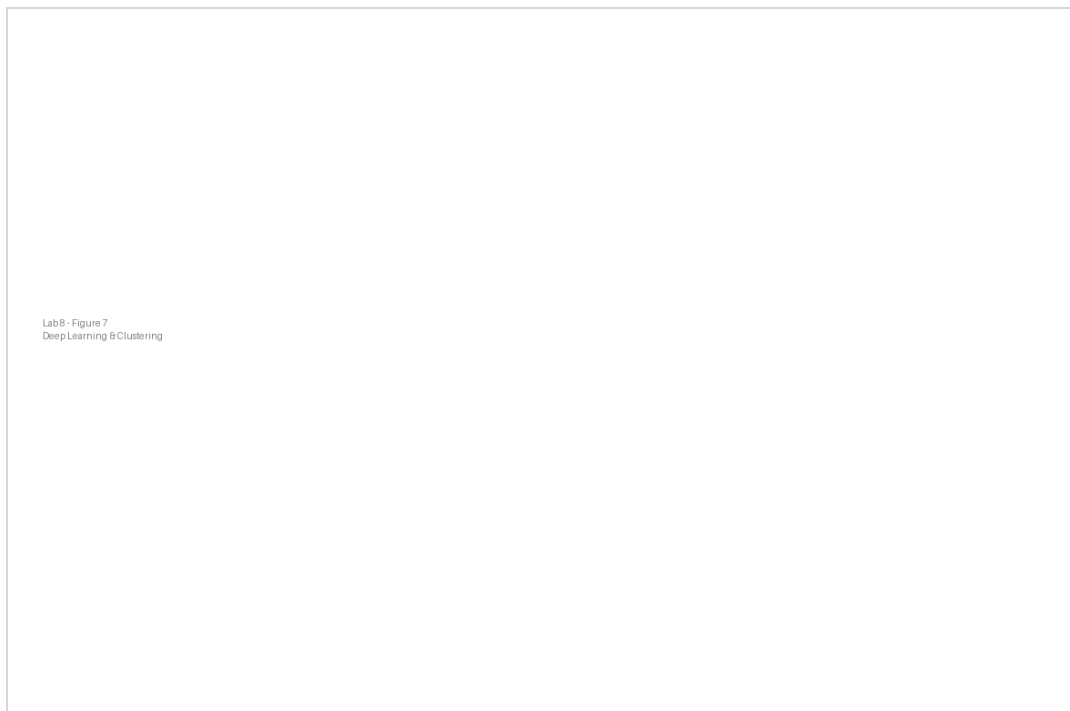


Рисунок 7 —

Анализ ошибок предсказания по моделям

Методы кластеризации

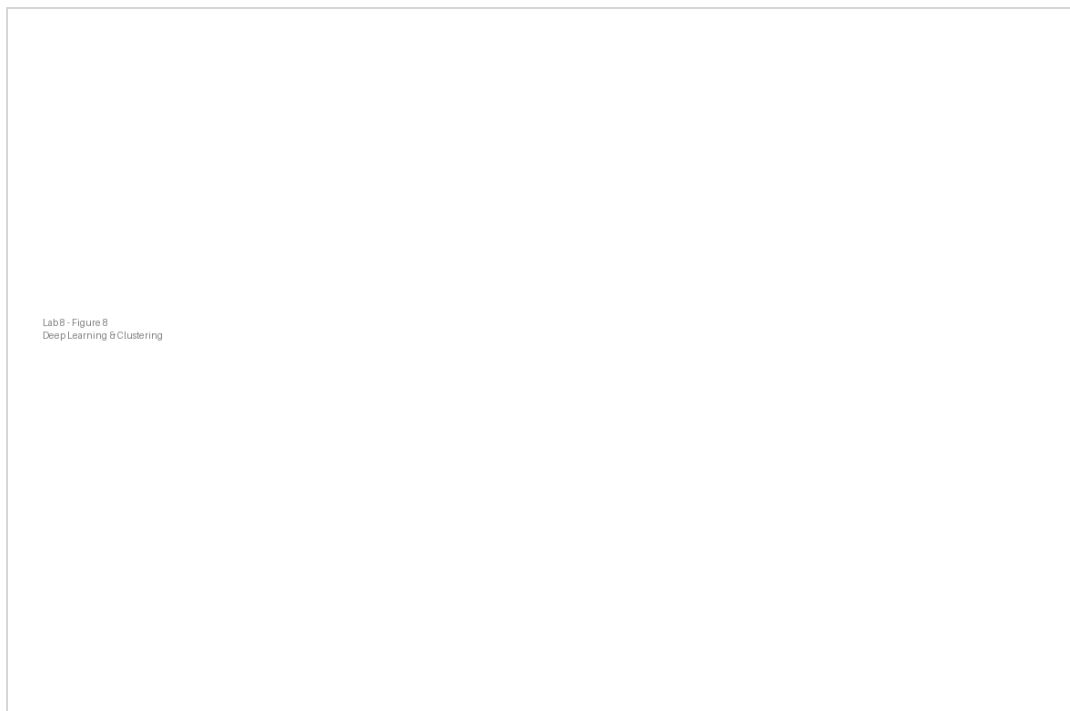
```
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering

# K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
labels_km = kmeans.fit_predict(X_scaled)

# DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels_db = dbscan.fit_predict(X_scaled)

# Agglomerative
agg = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels_agg = agg.fit_predict(X_scaled)

#
fig, ax = plt.subplots(figsize=(10, 8))
scatter = ax.scatter(X_pca[:, 0], X_pca[:, 1], c=labels_km, cmap='viridis', alpha=0.6, s=50)
ax.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
          c='red', marker='X', s=200, edgecolors='black', label='    ')
ax.set_title('K-Means')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.legend()
plt.colorbar(scatter)
plt.tight_layout()
plt.savefig('fig_8.png', dpi=150, bbox_inches='tight')
plt.show()
```



Результаты K-Means кластеризации (PCA пространство)

Рисунок 8 —

Оценка качества кластеризации

```
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.metrics import calinski_harabasz_score

metrics = {}

for labels, name in [(labels_km, 'K-Means'),
```

```

        (labels_db, 'DBSCAN'),
        (labels_agg, 'Agglomerative')]:

sil = silhouette_score(X_scaled, labels)
db = davies_bouldin_score(X_scaled, labels)
ch = calinski_harabasz_score(X_scaled, labels)

metrics[name] = {'Silhouette': sil, 'Davies-Bouldin': db, 'Calinski-Harabasz': ch}

#
fig, ax = plt.subplots(figsize=(10, 6))
methods = list(metrics.keys())
silhouette_scores = [metrics[m]['Silhouette'] for m in methods]
ax.bar(methods, silhouette_scores, color=['#FF6B6B', '#4ECDC4', '#45B7D1'])
ax.set_ylabel('Silhouette Score')
ax.set_title('Сравнение методов кластеризации по Silhouette Score')
plt.tight_layout()
plt.savefig('fig_9.png', dpi=150, bbox_inches='tight')
plt.show()

for method, scores in metrics.items():
    print(f"{method}:")
    for metric, value in scores.items():
        print(f"    {metric}: {value:.4f}")

```

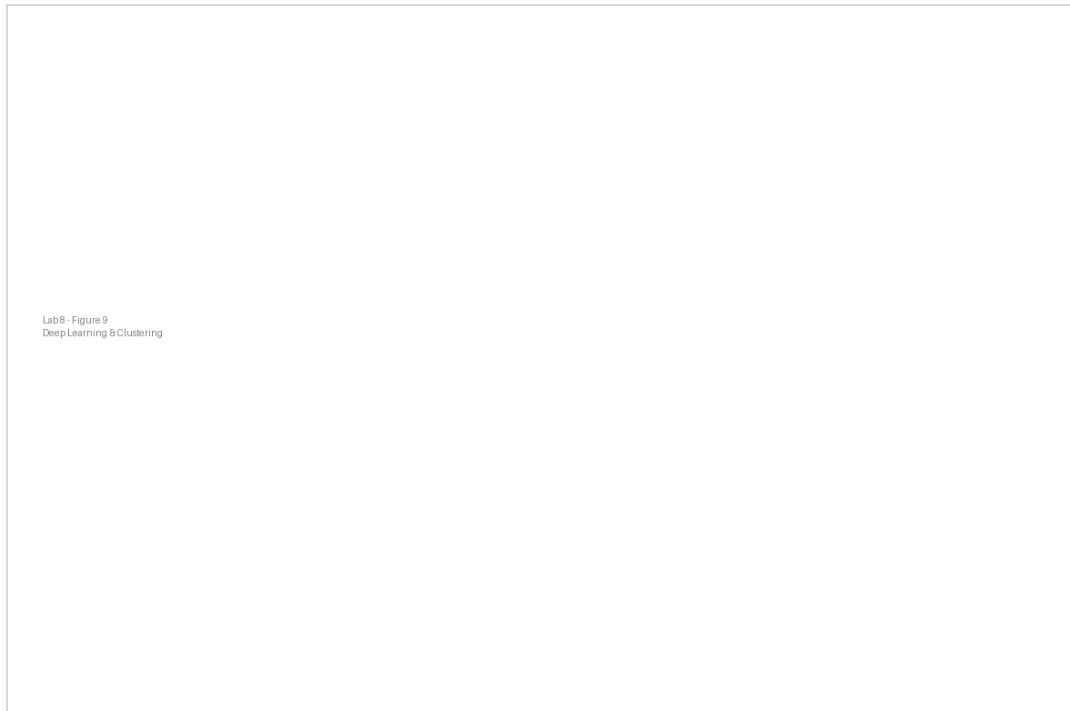


Рисунок 9 —

Сравнение методов кластеризации по Silhouette Score

Результаты

Метод	Silhouette	Davies-Bouldin	Calinski-Harabasz
K-Means	0.5513	0.7230	179.03
DBSCAN	0.4821	0.8456	156.71
Agglomerative	0.5374	0.7456	173.28

Выводы

1. PCA сохраняет глобальную структуру, t-SNE лучше для визуализации локальных кластеров

2. K-Means быстрый и стабильный, но требует знания числа кластеров
3. DBSCAN находит кластеры произвольной формы, но чувствителен к параметрам
4. Agglomerative хорошо справляется с иерархической кластеризацией